```
})
.then(res => {
if (res.ok) {
res.json().then(json => {
localStorage.setItem('token', json.token);
window.location.replace('/capability.html'); #A
});
}
})
.catch(error => console.error('Error logging in: ', error));
}
```

#A After login redirect to the capability-based message browser.

To complete the change, you now need to open the capability.js file again and adjust the `getCap` function to also send an X-CSRF-Token header on requests to the API, just as you previously did in the old Natter UI by extracting the token from `localStorage`. Listing 9.13 shows the updated function. If you now restart the API and create a demo user and space, you can then open a browser to https://localhost:4567/login.html. Type in the username and password of your demo user and you will be redirected to the message browser with a session cookie. You can now load any messages in the space and see that the audit log shows the correct username for each request.

Listing 9.13 Populating the X-CSRF-Header

```
function getCap(url, callback) {
let capUrl = new URL(url);
let token = capUrl.username;
capUrl.username = '';
return fetch(capUrl.href, {
```